ГУАП

КАФЕДРА № 44

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доц, к.т.н., доц.		А.Н. Долидзе
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

ПРОГРАММНАЯ РЕАЛИЗАЦИЯ АЛГОРИТМА ВЫПОЛНЕНИЯ ЦЕЛОЧИСЛЕННОЙ ОПЕРАЦИИ ДЛЯ АРХИТЕКТУРЫ НАБОРА КОМАНД ARM

по курсу: Организация ЭВМ и вычислительных систем

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4341в	04.10.2026	Р.С. Кулаков
		подпись, дата	инициалы, фамилия

Санкт-Петербург
2026

1 Задание

Реализовать на языке ассемблера ARM алгоритм целочисленной операции, разработанный в лабораторной работе №1, с учётом дополнительной функции. Выбрать места хранения данных, перевести блоки алгоритма в команды ARM, отладить программу в ARMSim# и представить результаты вычислений.

В соответствии с заданием ЛР1 для номера по журналу 11:

- Номер алгоритма: `11 mod 16 + 1 = 12`.\
- Номер дополнительной функции: `11 mod 10 + 1 = 2`.
- Число информационных разрядов операндов: `11 mod 6 + 4 = 9`.
- Полная разрядность каждого операнда с учётом знака: `9 + 1 = 10` бит.

Алгоритм №12: умножение целых чисел со знаком в дополнительном коде с неподвижной суммой частичных произведений, сдвигом множимого влево и анализом множителя начиная с младших разрядов, с предварительным изменением знака.

Дополнительная функция №2: переместить три младших разряда результата в другой регистр, оставшиеся разряды не перемещать. Как в ЛР1, это реализуется копированием `R := P AND 7` с сохранением исходного `P`.

Основной пример: `X = 13`, `Y = −7`. Ожидается `P = −91`, `R = 5 = 101₂`.

2 Список переменных и размещение данных

Рабочие переменные размещены в регистрах общего назначения, исходные числа и окончательные результаты — также в памяти. Регистры ARM имеют разрядность 32 бита. Отрицательные числа загружаются в дополнительном коде со знаковым расширением.

Таблица 1

Переменная	Назначение	Размещение
X, Y	Исходное множимое множитель	inputs, п вычислении r0, r1
S	Сумма частичн произведений	r2
M	Рабочее множим сдвигаемое влево	r3
Q	Неотрицательный рабоч множитель	r4
i	Число обработанн разрядов, 0…9	r5
P	Произведение	r6 и первое сло results
R	Три младших би произведения	r7 и второе сло results
—	Указатель чтения исходн данных	r8
—	Указатель запи результатов	r9

После вычисления `r0` и `r1` используются для параметров вывода. Значения `P` и `R` сохраняются в `r6`, `r7` и памяти. Трёхразрядный регистр из ЛР1 моделируется младшими тремя битами 32-разрядного `r7`; его старшие биты равны нулю.

В ЛР1 результат изображался 20-разрядным словом. Здесь он хранится в 32 битах. Для выбранных операндов переполнения нет; младшие 20 бит совпадают с представлением ЛР1.

3 Блок-схема алгоритма

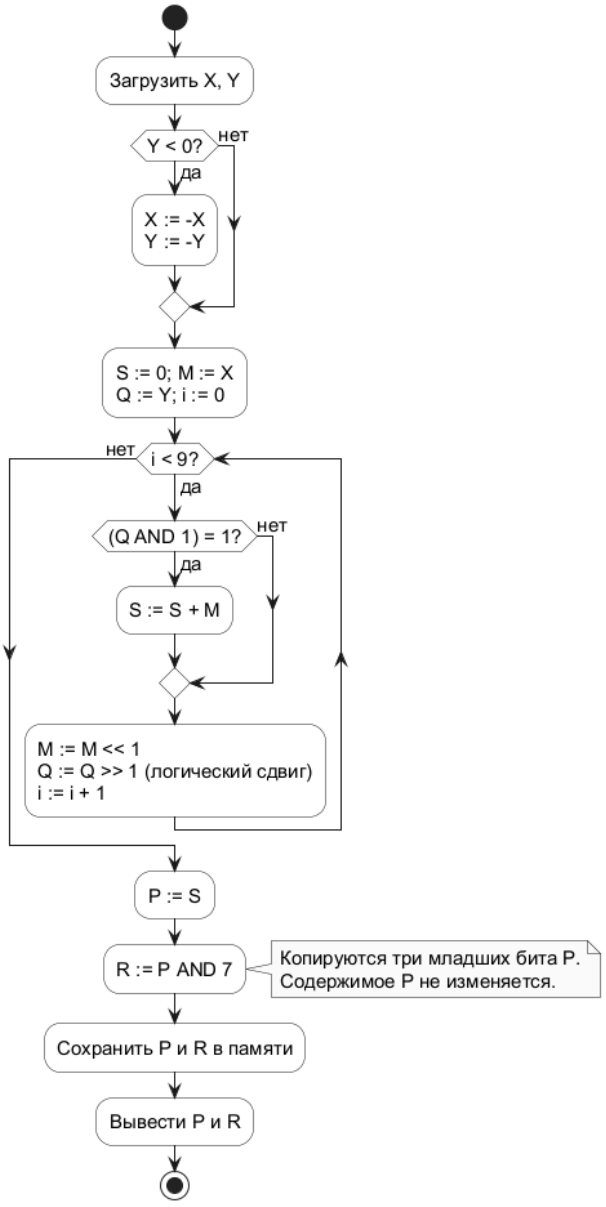


Рисунок 1 - Алгоритм умножения с предварительным изменением знаков операндов и выделением трёх младших разрядов результата

Если $Y < 0$, оба операнда меняют знак. Равенство $X \cdot Y = (-X) \cdot (-Y)$ сохраняет произведение. После этого $S = 0$, $M = X$, $Q = Y$, $i = 0$.

На каждой из девяти итераций проверяется младший бит Q. Если он равен единице, выполняется S := S + M. Затем M сдвигается влево, Q — логически вправо, а счётчик увеличивается. По окончании цикла выполняются P := S и R := P AND 7.

Для сохранённого из ЛР1 цикла из 9 итераций требуется |Y| ≤ 511. Граничное значение 10-разрядного дополнительного кода Y = −512 требует отдельной обработки или десятой итерации. Все выбранные примеры находятся в допустимой области алгоритма ЛР1.

4 Код программы на языке ассемблера ARM

Основная программа приведена ниже. Исходный файл — lab2.s.

```
@ Lab 2. Journal 11; algorithm 12; function 2; n = 9.
@ Main example: 13 * (-7) = -91; low three bits = 5.
        .text
        .global _start
_start:
        LDR r8, =inputs
        LDR r9, =results
        LDR r0, [r8], #4
        LDR r1, [r8], #4
        CMP r1, #0
        BGE init
        RSB r0, r0, #0
        RSB r1, r1, #0
init:
        MOV r2, #0
        MOV r3, r0
        MOV r4, r1
        MOV r5, #0
multiply_loop:
        CMP r5, #9
        BGE finish_product
        TST r4, #1
        BEQ shift
        ADD r2, r2, r3
shift:
        MOV r3, r3, LSL #1
        MOV r4, r4, LSR #1
        ADD r5, r5, #1
        B multiply_loop
finish_product:
        MOV r6, r2
        AND r7, r6, #7
        STR r6, [r9], #4
        STR r7, [r9], #4
        LDR r0, =label_p
        SWI 0x02
        MOV r0, #1
        MOV r1, r6
        SWI 0x6b
        LDR r0, =label_r
```

```
        SWI 0x02
        MOV r0, #1
        MOV r1, r7
        SWI 0x6b
        MOV r0, #10
        SWI 0x00
        SWI 0x11
        .data
        .align 2
inputs:
        .word 13, -7
results:
        .word 0, 0
label_p:
        .asciz "P = "
label_r:
        .asciz "; R = "
        .end
```

4.1 Соответствие блоков схемы командам

Таблица 2

Блок алгоритма	Реализация
Загрузка X, Y	LDR загружает адреса, затем значен слов памяти
Проверка Y < 0	CMP r1, #0; знаковый переход BGE init
Изменение знаков	RSB r0, r0, #0; RSB r1, r1, #0
Инициализация S, M, Q, i	Четыре команды MOV после метки init
Проверка i < 9	CMP r5, #9; BGE finish_product
Проверка младшего бита	TST r4, #1; BEQ shift при нулевом бите
Сложение S + M	ADD r2, r2, r3
Сдвиги и счётчик	MOV c LSL #1, MOV c LSR #1, ADD r5, #1
Сохранение P	MOV r6, r2
Дополнительная функция	AND r7, r6, #7, маска $7 = 111_2$
Запись результатов	Две команды STR
Вывод и завершение	Сервисы SWI симулятора

Команда `TST` вычисляет логическое И для установки флагов, не изменяя операнды. `BEQ` выполняет переход при `Z = 1`. Команда `RSB` с нулём вычисляет

отрицание операнда. `MOV` со сдвигом позволяет использовать сдвиговый механизм ARM. Команда аппаратного умножения `MUL` не применяется.

При адресации `[r8], #4` или `[r9], #4` сначала используется текущий адрес, затем указатель увеличивается на 4 байта. Директива `.word` задаёт 32-разрядные слова, `.asciz` — строку с завершающим нулём. `.end` завершает исходный текст; остановку выполнения обеспечивает `SWI 0x11`.

Сервисы ARMSim# 1.91: `SWI 0x02` выводит строку по адресу в `r0`; `SWI 0x6B` выводит целое число из `r1` при `r0 = 1`; `SWI 0x00` выводит символ из `r0`. Значение 10 соответствует переводу строки.

5 Карта распределения памяти

Адреса определены по листингу ARMSim# 1.91 для приведённой программы. В памяти используются 32-разрядные слова по 4 байта. Регистры процессора собственных адресов в этой карте не имеют.

Таблица 3

Диапазон адресов	Размер, байт	Назначение
0x1000–0x100F	16	Загрузка указателей операндов
0x1010–0x101F	16	Проверка и предварительн изменение знаков
0x1020–0x102F	16	Инициализация S, M, Q, i
0x1030–0x1053	36	Цикл обработки разрядов
0x1054–0x1063	16	Формирование P, R и запи результатов
0x1064–0x1097	52	Вывод и останов программы
0x1098–0x10A7	16	Пул литералов: адреса мет для LDR reg, =label
0x10A8–0x10AF	8	Исходные операнды inputs
0x10B0–0x10B7	8	Результаты results
0x10B8–0x10BC	5	Строка P = и нулевой бай
0x10BD–0x10C3	7	Строка ; R = и нулевой ба

Исполняемый код занимает 152 байта: 38 команд по 4 байта. Код, пул литералов и данные занимают вместе 196 байт (`0xC4`). Стек и динамическая память в программе не используются.

Таблица 4

Адрес слова	Содержимое	До запуска	После запуска
0x10A8	X	13 = 0x0000000D	Без изменения
0x10AC	Y	−7 = 0xFFFFFFF9	Без изменения
0x10B0	P	0	−91 = 0xFFFFFFA5
0x10B4	R	0	5 = 0x00000005

Пул литералов содержит `0x10A8` по адресу `0x1098`, `0x10B0` по `0x109C`, `0x10B8` по `0x10A0` и `0x10BD` по `0x10A4`. Эти слова создаются ассемблером для загрузки адресов меток.

6 Пример вычисления и проверка в симуляторе

Для исходных чисел `13` и `-7` выполняется предварительное изменение знаков: `X = -13`, `Y = 7`. Начальное состояние цикла: `S = 0`, `M = -13`, `Q = 7`, `i = 0`.

Таблица 5

Итерация	M до сдвига	Q до сдвига	Q_0	S после возможного сложения
1	−13	7	1	−13
2	−26	3	1	−39
3	−52	1	1	−91
4	−104	0	0	−91
5	−208	0	0	−91
6	−416	0	0	−91
7	−832	0	0	−91
8	−1664	0	0	−91
9	−3328	0	0	−91

После девятой итерации `M = -6656`, `Q = 0`, `i = 9`. Произведение равно `P = -91`. В 20-разрядном дополнительном коде это 11111111111110100101_2, в 32-разрядном — `0xFFFFFFA5`. Три младших бита: 101_2, поэтому `R = 5`. Значение P после маскирования сохраняется.

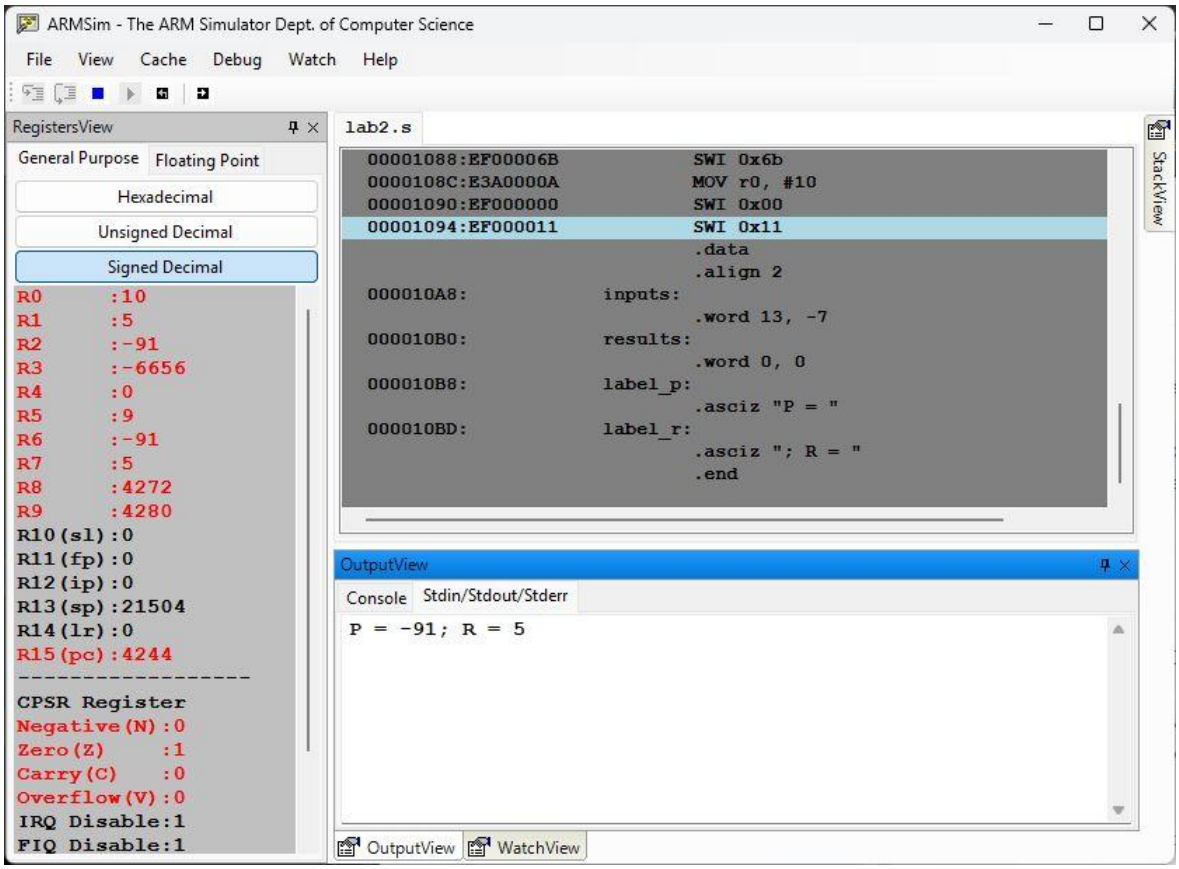


Рисунок 2 - Выполнение основной программы: r6 = −91, r7 = 5

Программа собрана и выполнена в ARMSim# 1.91. В консоли получено `P = -91; R = 5`. Выполнение завершилось сервисом `SWI 0x11`. Симулятор сообщил о выполнении 106 команд. На скриншоте `r8 = 4272 = 0x10B0`, `r9 = 4280 = 0x10B8`, что соответствует чтению двух операндов и записи двух результатов.

6.1 Дополнительные проверки из ЛР1

Файл check-five.s содержит внешний цикл для пяти пар исходных данных. Арифметическая часть и дополнительная функция совпадают с основной

программой. Этот файл имеет собственное размещение команд и данных; карта раздела 5 относится к `lab2.s`.

Таблица 6

X	Y	Ожидаемое P	Получено P	Получено R	R в двоичном виде
13	−7	−91	−91	5	101
−25	−6	150	150	6	110
21	13	273	273	1	001
−18	5	−90	−90	6	110
31	−9	−279	−279	1	001

7 Вывод

Изучены команды ARM для загрузки и сохранения данных, сложения, изменения знака, проверки битов, сдвигов и условных переходов. На их основе реализован алгоритм №12 из ЛР1 и дополнительная функция №2. Исходные операнды и результаты размещены в памяти, промежуточные величины — в регистрах. Составлены карта памяти и полная трассировка основного примера. Проверка в ARMSim# подтвердила результат `13 · (−7) = −91` и выделение младших трёх битов 101_2; дополнительные примеры с различными знаками также совпали с расчётами ЛР1.

8 Приложение А Полная пошаговая трассировка

Ниже приведена расчётная трассировка для `X = 13`, `Y = -7`. Она включает все девять проходов цикла, выход из цикла, запись результатов, вывод и остановку. Число шагов и итоговое состояние сверены с ARMSim#; таблица не является экспортом журнала симулятора.

В столбцах показаны записываемые регистры и ячейки памяти, адрес очередной команды `PC`, а также флаги `N`, `Z`, `V`, используемые переходами. Неиспользуемый флаг переноса `C` не приводится. Адреса `PC` записаны шестнадцатеричными числами. Здесь `PC` означает адрес выполняемой или следующей команды в листинге; при чтении `r15` внутри ARM-команды действует отдельное правило конвейерного смещения. Начальные значения `r0…r9`, `N`, `Z`, `V` равны нулю.

Таблица 7

Шаг	Команда	Было	Стало
1	LDR r8, =inputs	PC=1000; r8=0x0000	PC=1004; r8=0x10A8
2	LDR r9, =results	PC=1004; r9=0x0000	PC=1008; r9=0x10B0
3	LDR r0, [r8], #4	PC=1008; r0=0; r8=0x10A8	PC=100C; r0=13; r8=0x10AC
4	LDR r1, [r8], #4	PC=100C; r1=0; r8=0x10AC	PC=1010; r1=-7; r8=0x10B0
5	CMP r1, #0	PC=1010; N=0, Z=0, V=0	PC=1014; N=1, Z=0, V=0
6	BGE init	PC=1014	PC=1018; переход не выполнен
7	RSB r0, r0, #0	PC=1018; r0=13	PC=101C; r0=-13
8	RSB r1, r1, #0	PC=101C; r1=-7	PC=1020; r1=7
9	MOV r2, #0	PC=1020; r2=0	PC=1024; r2=0
10	MOV r3, r0	PC=1024; r3=0	PC=1028; r3=-13
11	MOV r4, r1	PC=1028; r4=0	PC=102C; r4=7
12	MOV r5, #0	PC=102C; r5=0	PC=1030; r5=0
13	CMP r5, #9	PC=1030; N=1, Z=0, V=0	PC=1034; N=1, Z=0, V=0
14	BGE finish_product	PC=1034	PC=1038; переход не выполнен
15	TST r4, #1	PC=1038; N=1, Z=0, V=0	PC=103C; N=0, Z=0, V=0

16	BEQ shift	PC=103C	PC=1040; переход не выполнен
17	ADD r2, r2, r3	PC=1040; r2=0	PC=1044; r2=-13
18	MOV r3, r3, LSL #1	PC=1044; r3=-13	PC=1048; r3=-26
19	MOV r4, r4, LSR #1	PC=1048; r4=7	PC=104C; r4=3
20	ADD r5, r5, #1	PC=104C; r5=0	PC=1050; r5=1
21	B multiply_loop	PC=1050	PC=1030; переход выполнен
22	CMP r5, #9	PC=1030; N=0, Z=0, V=0	PC=1034; N=1, Z=0, V=0
23	BGE finish_product	PC=1034	PC=1038; переход не выполнен
24	TST r4, #1	PC=1038; N=1, Z=0, V=0	PC=103C; N=0, Z=0, V=0
25	BEQ shift	PC=103C	PC=1040; переход не выполнен
26	ADD r2, r2, r3	PC=1040; r2=-13	PC=1044; r2=-39
27	MOV r3, r3, LSL #1	PC=1044; r3=-26	PC=1048; r3=-52
28	MOV r4, r4, LSR #1	PC=1048; r4=3	PC=104C; r4=1
29	ADD r5, r5, #1	PC=104C; r5=1	PC=1050; r5=2
30	B multiply_loop	PC=1050	PC=1030; переход выполнен
31	CMP r5, #9	PC=1030; N=0, Z=0, V=0	PC=1034; N=1, Z=0, V=0
32	BGE finish_product	PC=1034	PC=1038; переход не выполнен
33	TST r4, #1	PC=1038; N=1, Z=0, V=0	PC=103C; N=0, Z=0, V=0
34	BEQ shift	PC=103C	PC=1040; переход не выполнен
35	ADD r2, r2, r3	PC=1040; r2=-39	PC=1044; r2=-91
36	MOV r3, r3, LSL #1	PC=1044; r3=-52	PC=1048; r3=-104
37	MOV r4, r4, LSR #1	PC=1048; r4=1	PC=104C; r4=0
38	ADD r5, r5, #1	PC=104C; r5=2	PC=1050; r5=3
39	B multiply_loop	PC=1050	PC=1030; переход выполнен
40	CMP r5, #9	PC=1030; N=0, Z=0, V=0	PC=1034; N=1, Z=0, V=0
41	BGE finish_product	PC=1034	PC=1038; переход не выполнен
42	TST r4, #1	PC=1038; N=1, Z=0, V=0	PC=103C; N=0, Z=1, V=0
43	BEQ shift	PC=103C	PC=1044; переход выполнен

44	MOV r3, r3, LSL #1	PC=1044; r3=-104	PC=1048; r3=-208
45	MOV r4, r4, LSR #1	PC=1048; r4=0	PC=104C; r4=0
46	ADD r5, r5, #1	PC=104C; r5=3	PC=1050; r5=4
47	B multiply_loop	PC=1050	PC=1030; переход выполнен
48	CMP r5, #9	PC=1030; N=0, Z=1, V=0	PC=1034; N=1, Z=0, V=0
49	BGE finish_product	PC=1034	PC=1038; переход не выполнен
50	TST r4, #1	PC=1038; N=1, Z=0, V=0	PC=103C; N=0, Z=1, V=0
51	BEQ shift	PC=103C	PC=1044; переход выполнен
52	MOV r3, r3, LSL #1	PC=1044; r3=-208	PC=1048; r3=-416
53	MOV r4, r4, LSR #1	PC=1048; r4=0	PC=104C; r4=0
54	ADD r5, r5, #1	PC=104C; r5=4	PC=1050; r5=5
55	B multiply_loop	PC=1050	PC=1030; переход выполнен
56	CMP r5, #9	PC=1030; N=0, Z=1, V=0	PC=1034; N=1, Z=0, V=0
57	BGE finish_product	PC=1034	PC=1038; переход не выполнен
58	TST r4, #1	PC=1038; N=1, Z=0, V=0	PC=103C; N=0, Z=1, V=0
59	BEQ shift	PC=103C	PC=1044; переход выполнен
60	MOV r3, r3, LSL #1	PC=1044; r3=-416	PC=1048; r3=-832
61	MOV r4, r4, LSR #1	PC=1048; r4=0	PC=104C; r4=0
62	ADD r5, r5, #1	PC=104C; r5=5	PC=1050; r5=6
63	B multiply_loop	PC=1050	PC=1030; переход выполнен
64	CMP r5, #9	PC=1030; N=0, Z=1, V=0	PC=1034; N=1, Z=0, V=0
65	BGE finish_product	PC=1034	PC=1038; переход не выполнен
66	TST r4, #1	PC=1038; N=1, Z=0, V=0	PC=103C; N=0, Z=1, V=0
67	BEQ shift	PC=103C	PC=1044; переход выполнен
68	MOV r3, r3, LSL #1	PC=1044; r3=-832	PC=1048; r3=-1664
69	MOV r4, r4, LSR #1	PC=1048; r4=0	PC=104C; r4=0
70	ADD r5, r5, #1	PC=104C; r5=6	PC=1050; r5=7
71	B multiply_loop	PC=1050	PC=1030; переход выполнен

72	CMP r5, #9	PC=1030; N=0, Z=1, V=0	PC=1034; N=1, Z=0, V=0
73	BGE finish_product	PC=1034	PC=1038; переход не выполнен
74	TST r4, #1	PC=1038; N=1, Z=0, V=0	PC=103C; N=0, Z=1, V=0
75	BEQ shift	PC=103C	PC=1044; переход выполнен
76	MOV r3, r3, LSL #1	PC=1044; r3=-1664	PC=1048; r3=-3328
77	MOV r4, r4, LSR #1	PC=1048; r4=0	PC=104C; r4=0
78	ADD r5, r5, #1	PC=104C; r5=7	PC=1050; r5=8
79	B multiply_loop	PC=1050	PC=1030; переход выполнен
80	CMP r5, #9	PC=1030; N=0, Z=1, V=0	PC=1034; N=1, Z=0, V=0
81	BGE finish_product	PC=1034	PC=1038; переход не выполнен
82	TST r4, #1	PC=1038; N=1, Z=0, V=0	PC=103C; N=0, Z=1, V=0
83	BEQ shift	PC=103C	PC=1044; переход выполнен
84	MOV r3, r3, LSL #1	PC=1044; r3=-3328	PC=1048; r3=-6656
85	MOV r4, r4, LSR #1	PC=1048; r4=0	PC=104C; r4=0
86	ADD r5, r5, #1	PC=104C; r5=8	PC=1050; r5=9
87	B multiply_loop	PC=1050	PC=1030; переход выполнен
88	CMP r5, #9	PC=1030; N=0, Z=1, V=0	PC=1034; N=0, Z=1, V=0
89	BGE finish_product	PC=1034	PC=1054; переход выполнен
90	MOV r6, r2	PC=1054; r6=0	PC=1058; r6=-91
91	AND r7, r6, #7	PC=1058; r7=0	PC=105C; r7=5
92	STR r6, [r9], #4	PC=105C; [0x10B0]=0; r9=0x10B0	PC=1060; [0x10B0]=-91; r9=0x10B4
93	STR r7, [r9], #4	PC=1060; [0x10B4]=0; r9=0x10B4	PC=1064; [0x10B4]=5; r9=0x10B8
94	LDR r0, =label_p	PC=1064; r0=-13	PC=1068; r0=0x10B8
95	SWI 0x02	PC=1068	PC=106C; вывод «P = »
96	MOV r0, #1	PC=106C; r0=0x10B8	PC=1070; r0=1
97	MOV r1, r6	PC=1070; r1=7	PC=1074; r1=-91
98	SWI 0x6b	PC=1074	PC=1078; вывод числа -91
99	LDR r0, =label_r	PC=1078; r0=1	PC=107C; r0=0x10BD
100	SWI 0x02	PC=107C	PC=1080; вывод «; R = »

101	MOV r0, #1	PC=1080; r0=0x10BD	PC=1084; r0=1
102	MOV r1, r7	PC=1084; r1=-91	PC=1088; r1=5
103	SWI 0x6b	PC=1088	PC=108C; вывод числа 5
104	MOV r0, #10	PC=108C; r0=1	PC=1090; r0=10
105	SWI 0x00	PC=1090	PC=1094; перевод строки
106	SWI 0x11	PC=1094	PC=1098; остановка